

DUNIX

DARKSHELL OPERATOR PRIMER

A first terminal course for citizens of Civilization Y

CITY NETWORK // YANS // YEAR 2990

AUTHORIZED TRAINING COPY

For archive clerks, maintenance apprentices, field technicians, and any citizen touching a terminal for the first time.

```
$ o
/home/apprentice
$ l
notes/ orders.txt sector-map tools/
$ _
```

CONTINUITY SERIES 01

Read this first

Dunix is the operating system that keeps the City alive. It runs archive terminals, hydroponic controls, transit gates, weather relays, medical stations, workshop machines, and the Yotta Advanced Network System. Darkshell - written as dsh - is the command environment through which a citizen speaks to Dunix.

This primer assumes that you have never used a Unix-like system. It begins with the prompt and ends with practical text processing, process control, permissions, networking, encryption, and a complete field exercise. Every chapter follows the same discipline: observe, enter one command, inspect the result, then combine commands.

PRIME DIRECTIVE

Do not type a command merely because it appears in a manual. First identify the object it will affect. Commands that remove files, change ownership, stop processes, or operate with elevated authority are marked clearly.

How commands are shown

Terminal sessions appear in dark panels. The dollar sign is the prompt. You do not type the prompt itself. Type only the text after it, then press Enter.

TERMINAL SESSION

```
$ p READY
READY
```

Words in angle brackets are placeholders. Replace them with real names; do not type the brackets. For example, the pattern `cp <source> <destination>` becomes `cp report.txt report.bak`.

The Dunix habit

- Use small commands that do one job well.
- Keep information as plain text whenever possible.
- Inspect before changing; make a copy before experimenting.
- Combine commands with pipes instead of searching for one enormous program.
- Prefer reproducible command sequences over undocumented manual actions.

The method is descended from the classic programming-environment tradition associated with Brian W. Kernighan and Rob Pike: learn the system by using it, and build larger operations by composing small tools. This manual is an original Civilization Y training text; it does not reproduce that earlier work.

Course map

COMMAND	PURPOSE	PATTERN	EXAMPLE
01-03	Terminal, directories, files	o l cd md mcd	learn where you are
04-06	Editing and reading text	v c h t le b	create and inspect notes
07-09	Streams and filters	> g sort awk	turn text into reports
10-12	Storage, processes, authority	tar ps top chm sd	operate a live system
13-15	Programs, networks, protection	cc png s enc	diagnose and secure
16	Final field exercise	combined workflow	restore a station report

CHAPTER 01

The terminal and the prompt

Your first conversation with a Unix machine

A terminal is a text channel between you and Unix. The machine prints a prompt when it is ready. A typical prompt contains the terminal host name, and may also show the active development branch. The final \$ means that you are operating with ordinary citizen authority.

FIRST COMMAND

```
archive-07 $ p hello
hello
archive-07 $
```

The command is p. Its argument is hello. Unix runs the command, prints the result, and returns to the prompt.

Command structure

Most commands follow one pattern: command, options, arguments. Spaces separate the parts.

STRUCTURE EXAMPLES

```
$ l -l records
$ cp -i report.txt archive/report.txt
```

- command: the operation to perform, such as l or cp
- option: a behavior switch, usually beginning with -, such as -l or -i
- argument: the object to inspect or change, such as records or report.txt

Correcting a line

Darkshell uses the v editing discipline on the command line. While entering ordinary commands, Backspace removes the previous character. Left and Right move through the line. Up recalls earlier commands. Ctrl-L clears the screen without erasing history.

DO NOT PANIC

A terminal usually does exactly what you ask, not what you intended. When the result surprises you, stop, read the message, and inspect your location with o.

Essential session commands

COMMAND	PURPOSE	PATTERN	EXAMPLE
p	print text	p <text>	p SECTOR READY
date	show date and time	date	date
id	show your identity and groups	id	id
un	identify the Unix kernel and host	un	un
u	show uptime and system load	u	u
cl	clear the terminal display	cl	cl
x	leave the current shell session	x	x

FIRST CONTACT DRILL

Complete the sequence exactly as written. Read each result before continuing.

1. Enter p CITIZEN LINK ESTABLISHED.
2. Enter date and read the current station time.
3. Enter id and locate your user name.
4. Enter o. You will learn what the result means in the next chapter.
5. Enter cl, then use the Up key to recall o.

CHAPTER 02

Where you are

Directories, paths, and safe movement

Dunix stores files inside directories. Directories form one tree. The top is written /. Your personal working area is normally under /home. At every moment, the shell has a current directory. Relative file names are interpreted from that location.

Print your location with o

CURRENT DIRECTORY

```
$ o
/home/apprentice
```

The result is an absolute path: a route beginning at /. The final component, apprentice, is the current directory.

List the directory with l

DIRECTORY LISTING

```
$ l
briefing.txt  logs/  projects/  tools*
```

A trailing / marks a directory. A trailing * marks an executable file. Use l -a to include hidden entries and l -l for ownership, permissions, size, and modification time.

LONG LISTING

```
$ l -la
drwxr-xr-x  6 apprentice ops 4096 Jul 24 08:40 ./
drwxr-xr-x 18 root      root 4096 Jul 20 13:10 ../
-rw-r--r--  1 apprentice ops  214 Jul 24 08:37 briefing.txt
```

Change directory with cd

COMMAND	PURPOSE	PATTERN	EXAMPLE
cd logs	enter a child directory	cd <directory>	cd logs
..	move to the parent	..	..
cd	return to your home directory	cd	cd
cd /var/log	use an absolute path	cd <absolute-path>	cd /var/log
cd -	return to the previous directory	cd -	cd -

MOVEMENT

```
$ o
/home/apprentice
$ cd projects
$ o
/home/apprentice/projects
$ ..
$ o
/home/apprentice
```

Create directories

Use `md` to create a directory. Use `md -p` for a complete path whose parent directories may not yet exist. Use `mcd` when you want to create a directory and enter it immediately.

CREATE AND ENTER

```
$ md reports
$ mcd projects/weather
$ o
/home/apprentice/projects/weather
```

NAME DISCIPLINE

Use short, meaningful names. Avoid spaces in operational file names. Prefer `reactor-log.txt` to `Reactor Log Final New.txt`.

NAVIGATION DRILL

Complete the sequence exactly as written. Read each result before continuing.

6. Return home with `cd`.
7. Create training with `md training`.
8. Enter it with `cd training` and confirm with `o`.
9. Create `labs/day1` and enter it in one step with `mcd labs/day1`.
10. Return to training using `...` Confirm with `o`.

CHAPTER 03

Files: create, copy, move, remove

Treat names as handles to stored information

A file is a named sequence of bytes. Many Unix files contain text, but a file may also hold a program, an image, an archive, device data, or encrypted material. Most commands accept file names as arguments.

Create an empty file

TOUCH

```
$ touch station.log
$ ls -l station.log
-rw-r--r-- 1 apprentice ops 0 Jul 24 09:02 station.log
```

touch creates an empty file when the name does not exist. If the file already exists, touch updates its modification time without changing its contents.

Copy with cp

COPY

```
$ cp station.log station.log.bak
$ cp -i report.txt archive/report.txt
```

The first argument is the source; the second is the destination. Use -i during training so Unix asks before overwriting an existing destination. Use cp -r to copy a directory tree.

Move or rename with mv

MOVE AND RENAME

```
$ mv station.log sector7.log
$ mv -i sector7.log logs/
```

mv changes a name or location. Moving within one storage system is normally quick because Unix updates directory information rather than copying every byte.

Remove carefully

DESTRUCTIVE COMMAND

rm removes directory entries. There is normally no recovery bin. Check o and l before using it. Use rm -i while learning.

INTERACTIVE REMOVAL

```
$ rm -i station.log.bak
rm: remove regular empty file 'station.log.bak'? y
```

rd removes an empty directory. rm -r removes a directory tree and everything beneath it; this is a high-risk operation and is intentionally excluded from the beginner drills.

Inspect before acting

COMMAND	PURPOSE	PATTERN	EXAMPLE
file	identify stored data	file <name>	file sector-map
stat	show detailed metadata	stat <name>	stat report.txt
size	show sections of an executable	size <program>	size relay
strings	extract readable text from binary data	strings <file>	strings relay h
tree	show a directory hierarchy	tree [directory]	tree training

Links

ln creates another directory entry. A hard link names the same stored data. ln -s creates a symbolic link that points to another path. Symbolic links are useful for stable names such as current pointing to releases/12.

SYMBOLIC LINK

```
$ ln -s releases/12 current
$ l -l current
current@ -> releases/12
```

FILE HANDLING DRILL

Complete the sequence exactly as written. Read each result before continuing.

11. Inside training, create empty note.txt with touch.
12. Copy it to note.txt.bak using cp -i.
13. Rename note.txt to field-note.txt using mv.
14. Inspect both files with file and stat.
15. Remove note.txt.bak using rm -i.

CHAPTER 04

Writing with v

The standard Dunix screen editor

v edits text inside the terminal. It is modal: the same key can insert a letter or issue a command depending on the current mode. This feels unusual for several minutes and efficient for the rest of your working life.

Open or create a file

START EDITOR

```
$ v field-note.txt
```

v opens the file. If it does not exist, the editor begins with a new empty buffer. The file is not created on storage until you write it.

The two modes you need first

COMMAND	PURPOSE	PATTERN	EXAMPLE
Normal	movement and editing commands	press Esc to return	Esc
Insert	type ordinary text	press i from Normal mode	i
:	enter a command-line action	press : from Normal mode	:w

Your first edit

16. Start v field-note.txt.
17. Press i. You are now in Insert mode.
18. Type: Sector 12 air filters inspected.
19. Press Esc to return to Normal mode.
20. Type :w and press Enter to write the file.
21. Type :q and press Enter to leave v.

INSIDE V AFTER WRITING

```
Sector 12 air filters inspected.
~
~
"field-note.txt" 1L, 33B
```

Minimum survival keys

COMMAND	PURPOSE	PATTERN	EXAMPLE
i	insert before cursor	Normal mode	i
a	insert after cursor	Normal mode	a
Esc	return to Normal mode	from Insert mode	Esc
h j k l	move left, down, up, right	Normal mode	j
x	delete character under cursor	Normal mode	x
dd	delete current line	Normal mode	dd

COMMAND	PURPOSE	PATTERN	EXAMPLE
u	undo last change	Normal mode	u
/word	search forward	Normal mode	/pressure
n	repeat search	Normal mode	n
:w	write file	command line	:w
:q	quit when unchanged	command line	:q
:wq	write and quit	command line	:wq
:q!	quit and discard changes	command line	:q!

RECOVERY RULE

When uncertain which mode you are in, press Esc twice. This is harmless and returns you to Normal mode.

A useful editing cycle

EDIT-SEARCH-SAVE

```
$ v checks.txt
# press i and enter several lines
# press Esc
:w
/filter
n
:wq
```

EDITOR DRILL

Complete the sequence exactly as written. Read each result before continuing.

22. Create checks.txt with v.
23. Enter three lines describing an inspection.
24. Write the file without leaving using :w.
25. Search for a word on the second line.
26. Delete one entire line with dd, then restore it with u.
27. Write and leave with :wq.

CHAPTER 05

Reading text

Choose the viewer that matches the amount of information

Small files can be displayed in full. Large files should be sampled or viewed page by page. The correct reading command prevents important lines from scrolling away.

Display a complete file with c

COMPLETE FILE

```
$ c field-note.txt
Sector 12 air filters inspected.
```

c writes the contents of one or more files to the terminal. It is best for short text. Multiple file names are displayed in the order given.

MULTIPLE FILES

```
$ c header.txt body.txt
```

Read the beginning or end

COMMAND	PURPOSE	PATTERN	EXAMPLE
h file	first 10 lines	h <file>	h station.log
h -30 file	first 30 lines	h -30 <file>	h -30 station.log
t file	last 10 lines	t <file>	t station.log
t -50 file	last 50 lines	t -50 <file>	t -50 station.log
t -f file	follow new lines as written	t -f <file>	t -f sensor.log

Stop a continuously running command such as t -f by pressing Ctrl-C. Ctrl-C requests that the foreground command stop; it does not close the terminal.

Page through long text

le opens a plain paged viewer. b opens an enhanced formatted viewer suited to source code and structured text. In either viewer, Space moves forward, b moves back, /word searches, n repeats the search, and q leaves.

PAGED VIEWERS

```
$ le /var/log/messages
$ b control.c
```

Count and number

COMMAND	PURPOSE	PATTERN	EXAMPLE
wc -l	count lines	wc -l <file>	wc -l citizens.txt
wc -w	count words	wc -w <file>	wc -w briefing.txt
wc -c	count bytes	wc -c <file>	wc -c packet.bin
nl	number lines	nl <file>	nl checklist.txt

COMMAND	PURPOSE	PATTERN	EXAMPLE
<code>rev</code>	reverse characters on each line	<code>rev <file></code>	<code>rev codes.txt</code>
<code>tac</code>	reverse line order	<code>tac <file></code>	<code>tac events.txt</code>

READING DRILL

Complete the sequence exactly as written. Read each result before continuing.

28. Display checks.txt with c.
29. Display only its first two lines with h -2.
30. Number its lines with nl.
31. Count its lines and words with wc.
32. Open it in le and leave with q.

CHAPTER 06

Streams and redirection

Make command output become input, files, and reports

Every command operates through streams. Standard input carries data into a command. Standard output carries normal results. Standard error carries diagnostics. The shell can connect these streams to files or other commands.

Write output to a file

OVERWRITE REDIRECTION

```
$ p "filter pressure normal" > status.txt
$ c status.txt
filter pressure normal
```

> creates the destination or replaces its previous contents. >> appends without erasing what is already there.

APPEND REDIRECTION

```
$ date >> status.txt
$ p "inspection complete" >> status.txt
$ c status.txt
```

OVERWRITE HAZARD

> does not ask before replacing a file. Confirm the destination name with ! before pressing Enter.

Use a file as input

INPUT REDIRECTION

```
$ wc -l < status.txt
3
```

Many commands can also accept a file name directly. Input redirection becomes especially useful when a command is designed to read from its standard input.

Connect commands with a pipe

The vertical bar | sends the standard output of the command on its left into the standard input of the command on its right. No temporary file is required.

PIPELINES

```
$ c citizens.txt | sort
$ ps -ef | g relay
$ c events.txt | g ERROR | wc -l
```

Read a pipeline from left to right: produce data, select or transform it, then summarize or store it.

See and save with tee

TEE

```
$ c events.txt | g ALERT | tee alerts.txt
```

```
2990-207 ALERT coolant variance
2990-207 ALERT gate timeout
```

tee copies its input both to the terminal and to a file. Use tee -a to append.

Quoting

Spaces separate arguments, so quote text that must remain one argument. Single quotes preserve text literally. Double quotes preserve spaces while still allowing shell variables to be expanded.

QUOTED ARGUMENTS

```
$ p 'Sector 4: stable'
Sector 4: stable
$ touch "shift report.txt"
```

OPERATIONAL NAMING

Although quoting supports spaces, operational files are easier to automate when their names contain letters, digits, -, _, and . only.

STREAM DRILL

Complete the sequence exactly as written. Read each result before continuing.

33. Create three lines in readings.txt using p with > for the first and >> for the next two.
34. Display readings.txt with c.
35. Pipe it through nl.
36. Pipe it through wc -l.
37. Use tee to copy it to readings.copy while also displaying it.

CHAPTER 07

Finding and selecting text

Locate names, lines, fields, and patterns

Find files with f

`f` walks a directory tree and reports matching paths. Start from a directory, then give tests such as `-name`, `-type`, or `-mtime`.

FIND PATHS

```
$ f . -name "*.log"
./logs/air.log
./logs/water.log
$ f /var/tmp -type f -mtime +7
```

START NARROW

A search from `/` may cross the entire machine and encounter restricted areas. Begin from `.` or a known directory.

Select lines with g

LINE SELECTION

```
$ g ERROR station.log
2990-207T08:14 ERROR valve-3 no response
$ g -i pressure readings.txt
$ g -v OK checks.txt
```

COMMAND	PURPOSE	PATTERN	EXAMPLE
<code>g pattern file</code>	lines containing pattern	<code>g <pattern> <file></code>	<code>g ALERT events.txt</code>
<code>g -i</code>	ignore letter case	<code>g -i <pattern> <file></code>	<code>g -i fault events.txt</code>
<code>g -n</code>	include line numbers	<code>g -n <pattern> <file></code>	<code>g -n ERROR events.txt</code>
<code>g -v</code>	lines not matching	<code>g -v <pattern> <file></code>	<code>g -v OK checks.txt</code>
<code>g -r</code>	search a directory tree	<code>g -r <pattern> <dir></code>	<code>g -r valve configs/</code>

Simple wildcard names

The shell expands wildcard characters before starting a command. `*` matches any sequence of characters. `?` matches one character. `[abc]` matches one listed character.

WILDCARDS

```
$ l *.txt
briefing.txt  checks.txt  status.txt
$ cp logs/2990-207-??log archive/
```

Locate commands

COMMAND	PURPOSE	PATTERN	EXAMPLE
<code>wh</code>	show the executable path chosen	<code>wh <command></code>	<code>wh awk</code>

COMMAND	PURPOSE	PATTERN	EXAMPLE
	by the shell		
whereis	locate program, source, and manual paths	whereis <name>	whereis awk
ty	report how the shell resolves a command name	ty <command>	ty cp
m	open a system manual page	m <long-command>	m awk

SEARCH DRILL

Complete the sequence exactly as written. Read each result before continuing.

38. Use `f . -name "*.txt"` in your training directory.
39. Use `g -n` filter checks.txt.
40. Use `g -v` inspected checks.txt and read what remains.
41. Use `l *.txt` to see shell wildcard expansion.
42. Use `wh awk` and `whereis awk`.

CHAPTER 08

Text filters

Sort, combine, cut, translate, and summarize

A filter reads text, transforms or selects it, and writes text. Filters become powerful when chained. The data remains visible at every step, so a result can be inspected and reproduced.

Sort and remove duplicates

SORT AND UNIQ

```
$ c sectors.txt
delta
alpha
delta
beta
$ c sectors.txt | sort | uniq
alpha
beta
delta
```

uniq removes adjacent duplicate lines, so sort usually comes first. uniq -c also counts repetitions. sort -n compares numbers numerically; sort -r reverses the order.

Select columns

CUT FIELDS

```
$ c crew.csv
A17,Irina,reactor
B04,Malik,water
$ cut -d, -f2,3 crew.csv
Irina,reactor
Malik,water
```

cut chooses character positions or delimiter-separated fields. paste combines lines from files side by side. tr translates or removes characters.

PASTE AND TRANSLATE

```
$ paste names.txt sectors.txt
$ c lower.txt | tr a-z A-Z
$ c dirty.txt | tr -d "
"
```

Format and reverse

COMMAND	PURPOSE	PATTERN	EXAMPLE
fmt	wrap paragraphs to a useful width	fmt -w <n> <file>	fmt -w 60 report.txt
nl	number lines	nl <file>	nl orders.txt
rev	reverse each line	rev <file>	rev access-codes.txt
tac	reverse the line sequence	tac <file>	tac events.txt
paste	merge files by line	paste <a> 	paste ids names

Generate data

SEQUENCE AND SHUFFLE

```
$ seq 1 5
1
2
3
4
5
$ seq 1 100 | shuf | h -1
47
```

seq generates numbered sequences. shuf randomizes input or chooses random numbers. These are useful for test data, sampling, and the traditional dsh dice commands.

Use xargs to build commands

XARGS

```
$ f reports -name "*.tmp" -print | xargs -n1 file
```

xargs converts input lines into command arguments. File names containing spaces or unusual characters require null-delimited forms; beginners should first use xargs only on controlled operational names.

FILTER DRILL

Complete the sequence exactly as written. Read each result before continuing.

43. Create a file containing alpha, delta, alpha, beta on separate lines.
44. Run sort on it.
45. Run sort | uniq -c.
46. Translate the output to uppercase with tr a-z A-Z.
47. Generate numbers 10 through 20 with seq and choose one with shuf | h -1.

CHAPTER 09

sed and awk

Small programs for text that has structure

When one option is not enough, Unix provides compact text languages. `sed` edits a stream line by line. `awk` divides lines into fields, evaluates conditions, and prints selected results. Both are designed for pipelines.

sed substitutions

SED

```
$ p 'VALVE=closed' | sed 's/closed/open/'
VALVE=open
$ sed 's/^/SECTOR-7 /' alarms.txt
```

The expression `s/old/new/` replaces the first old text on each line. Add `g` after the final slash to replace every occurrence. `sed` normally writes the changed text to standard output; the original file remains unchanged.

COMMAND	PURPOSE	PATTERN	EXAMPLE
<code>sed 's/a/b/'</code>	replace first match per line	<code>sed 's/<old>/<new>/'</code>	<code>sed 's/off/on/' state</code>
<code>sed 's/a/b/g'</code>	replace all matches per line	<code>sed 's/<old>/<new>/g'</code>	<code>sed 's/ /_/g' names</code>
<code>sed -n '5,10p'</code>	print selected lines	<code>sed -n '<range>p'</code>	<code>sed -n '20,30p' log</code>
<code>sed '/pat/d'</code>	delete matching lines from output	<code>sed '/<pattern>/d'</code>	<code>sed '/DEBUG/d' log</code>

awk fields and conditions

`awk` names fields `$1`, `$2`, and so on. `$0` is the complete input line. Its default field separator is runs of spaces or tabs.

AWK CONDITION

```
$ c pressure.tbl
sector value state
A 94 OK
B 121 ALERT
C 88 OK
$ awk '$2 > 100 {print $1, $2}' pressure.tbl
B 121
```

AWK TOTAL

```
$ awk 'NR > 1 {sum += $2} END {print sum}' pressure.tbl
303
```

`NR` is the current record number. `BEGIN` runs before input. `END` runs after input. Use `-F`, to read comma-separated fields.

AWK DELIMITER

```
$ awk -F, '{print $2, $3}' crew.csv
Irina reactor
Malik water
```

Build reports in stages

COMPOSED REPORT

```
$ g ALERT station.log | awk '{print $1, $4}' | sort | uniq -c | sort -nr
  7 valve-3
  2 pump-9
```

PIPELINE METHOD

Build a long pipeline one stage at a time. Run the first command, inspect it, add the second, inspect again, then continue.

STRUCTURED TEXT DRILL

Complete the sequence exactly as written. Read each result before continuing.

48. Create pressure.tbl with a heading and three data rows.
49. Use awk to print the first and second fields.
50. Use awk with a numeric condition to select high readings.
51. Use sed to replace ALERT with CHECK in the displayed output.
52. Redirect the transformed result to pressure-review.txt, then inspect it with c.

CHAPTER 10

Storage and archives

Measure space, package trees, and divide large data

Measure filesystems and directories

COMMAND	PURPOSE	PATTERN	EXAMPLE
<code>df -h</code>	free and used filesystem space	<code>df -h</code>	<code>df -h</code>
<code>du -sh</code>	total size of a path	<code>du -sh <path></code>	<code>du -sh logs</code>
<code>du -h -d1</code>	one level of directory sizes	<code>du -h -d1 <path></code>	<code>du -h -d1 /var</code>
<code>stat</code>	exact file metadata	<code>stat <file></code>	<code>stat archive.tgz</code>

`df` answers “how full is the storage device?” `du` answers “which paths consume the space?” Use both when investigating capacity.

Create a tar archive

CREATE ARCHIVE

```
$ tar -cvf shift-207.tar reports logs
reports/
reports/air.txt
logs/
logs/events.log
```

`tar` collects files and directory structure into one archive. `-c` creates, `-v` lists names, and `-f` supplies the archive file. Use `tar -tf` to inspect and `tar -xvf` to extract.

INSPECT AND EXTRACT

```
$ tar -tf shift-207.tar | h
$ mcd restore-test
$ tar -xvf ../shift-207.tar
```

Compress with gzip

COMPRESS AND RESTORE

```
$ gzip shift-207.tar
$ l -lh shift-207.tar.gz
$ gzip -d shift-207.tar.gz
```

A common compact form creates and compresses in one step: `tar -czvf archive.tgz directory`. Extract with `tar -xzvf archive.tgz`.

Split and join large files

SPLIT AND REJOIN

```
$ split -b 100M sensor-image.bin part-
$ l part-*
part-aa part-ab part-ac
$ c part-* > sensor-image.restored.bin
```

split divides a file into pieces. Concatenating the pieces in name order reconstructs the original. Verify with a trusted checksum command when one is available on the station image.

EXTRACTION RULE

Never extract an unfamiliar archive directly into an important directory. Create an empty test directory with mcd and inspect the archive list first.

ARCHIVE DRILL

Complete the sequence exactly as written. Read each result before continuing.

53. Measure the size of training with `du -sh`.
54. Create training.tar from the directory.
55. List the archive with `tar -tf`.
56. Create restore-test and extract the archive there.
57. Compare the original and restored trees with `tree`.

CHAPTER 11

Processes and time

Observe running work and control only what you own

A process is a running program. Each process has a numeric process identifier, or PID. Unix schedules many processes at once, records their ownership, and exposes their state as text.

Inspect the system

COMMAND	PURPOSE	PATTERN	EXAMPLE
<code>ps</code>	processes attached to this terminal	<code>ps</code>	<code>ps</code>
<code>ps -ef</code>	system-wide process list	<code>ps -ef</code>	<code>ps -ef g relay</code>
<code>top</code>	interactive live process view	<code>top</code>	<code>top</code>
<code>to</code>	one compact top snapshot	<code>to</code>	<code>to</code>
<code>free -h</code>	memory use	<code>free -h</code>	<code>free -h</code>
<code>u</code>	uptime and load averages	<code>u</code>	<code>u</code>
<code>w</code>	logged-in users and their work	<code>w</code>	<code>w</code>

Foreground and interruption

A command normally runs in the foreground and owns the terminal until it finishes. Ctrl-C requests termination. Ctrl-Z suspends a foreground command; the shell can later resume it with job-control commands available on full station images.

Delay, measure, continue

TIME AND BACKGROUND WORK

```
$ sleep 5
$ time sort large-list.txt > sorted.txt
$ nohup long-scan > scan.log 2>&1 &
```

`sleep` waits. `time` reports elapsed and CPU time. `nohup` allows a command to continue after terminal disconnection; `&` starts it in the background. Redirect output so a background process does not compete with your prompt.

Watch repeated output

REPEAT WITH WA

```
$ wa df -h
$ wa "ps -ef | g relay"
```

`wa` re-runs a command periodically and refreshes the display. Leave with Ctrl-C.

Stop a process

DESTRUCTIVE COMMAND

`z` sends a termination signal to a PID. Confirm the PID, owner, and command line with `ps` before acting. Never guess.

TERMINATE OWNED PROCESS

```
$ ps -ef | g stale-relay
apprentice 4182 ... stale-relay
$ z 4182
```

A normal termination request gives the process a chance to clean up. Stronger signals exist, but forcing termination can corrupt data and belongs to advanced incident procedure.

PROCESS DRILL

Complete the sequence exactly as written. Read each result before continuing.

58. Run `sleep 30` in the foreground and stop it with `Ctrl-C`.
59. Run `ps` and identify the shell process.
60. Run `free -h` and `u`.
61. Run `wa date`, observe two refreshes, and stop it with `Ctrl-C`.
62. Run `time seq 1 100000 | wc -l`.

CHAPTER 12

Permissions and authority

Who may read, write, execute, and administer

Every file has an owner, a group, and permission bits for owner, group, and others. The three basic permissions are read (r), write (w), and execute (x). Directories interpret these as listing, modification, and traversal rights.

PERMISSION STRING

```
$ l -l control.sh
-rwxr-x--- 1 irina reactor 814 Jul 24 07:41 control.sh
```

The first character describes the object type. The next three characters are owner permissions, then group permissions, then permissions for everyone else.

Change mode with chm

CHMOD PATTERNS

```
$ chm u+x control.sh
$ chm go-r secret.txt
$ chm 640 secret.txt
```

COMMAND	PURPOSE	PATTERN	EXAMPLE
u g o a	user, group, others, all	<class><op><perm>	chm g+w file
+ - =	add, remove, set exactly	<class><op><perm>	chm o-r file
r w x	read, write, execute	<class><op><perm>	chm u+x script
640	numeric: rw-r----	chm <mode> <file>	chm 640 secret.txt
755	numeric: rwxr-xr-x	chm <mode> <file>	chm 755 tool

Change ownership with cho

OWNERSHIP CHANGE

```
$ sd cho irina:reactor valve.conf
```

Changing ownership normally requires elevated authority. Use it only when a documented service account or group must own the file.

Elevated authority

sd runs the following command with administrative authority. sb opens an administrative shell. su changes user identity when policy permits.

RED PROMPT DISCIPLINE

Elevated authority removes many safety boundaries. Prefer one explicit sd command over a long sb session. Read the entire line before pressing Enter.

CONTROLLED ELEVATION

```
$ sd c /var/log/secure
$ sd chm 640 /etc/yans/relay.conf
```

```
$ sb  
# id  
# x
```

An administrative shell commonly displays # instead of \$. Leave it immediately after the required operation.

PERMISSION DRILL

Complete the sequence exactly as written. Read each result before continuing.

63. Create private.txt and inspect it with l -l.
64. Set mode 600 using chm 600 private.txt.
65. Create runme with v and place one line of text inside.
66. Add execute permission for yourself with chm u+x runme.
67. Inspect both permission strings and explain each triplet aloud.

CHAPTER 13

Numbers and programs

Use calculators, compilers, and inspection tools

Shell arithmetic with expr

EXPR

```
$ expr 7 + 5
12
$ expr 42 \* 3
126
```

expr requires spaces between arguments. The multiplication symbol must be protected from shell wildcard expansion.

Calculators bc and dc

CALCULATORS

```
$ p "scale=3; 17/6" | bc
2.833
$ p "16 8 / p" | dc
2
```

bc uses familiar algebraic notation and supports precision through scale. dc is a stack calculator: numbers are pushed first, then an operation is applied. Enter q to leave either interactive calculator.

Compile a small C program

cc translates C source into an executable program. Create hello.c with v:

HELLO.C

```
#include <stdio.h>

int main(void)
{
    printf("YANS link stable\n");
    return 0;
}
```

COMPILE AND RUN

```
$ cc -o hello hello.c
$ ./hello
YANS link stable
```

./ means “the file named hello in the current directory.” Unix does not normally search the current directory automatically for safety reasons.

Inspect programs

COMMAND	PURPOSE	PATTERN	EXAMPLE
file	identify executable format	file <program>	file hello

COMMAND	PURPOSE	PATTERN	EXAMPLE
size	show code and data section sizes	size <program>	size hello
ldd	show shared library requirements	ldd <program>	ldd hello
strings	show readable embedded strings	strings <program>	strings hello g YANS
od	display bytes in selected forms	od -Ax -tx1 <file>	od -Ax -tx1 hello h
gdb	debug a program	gdb <program>	gdb hello

Compare and patch

DIFFERENCE AND PATCH

```
$ d old.conf new.conf
$ patch < repair.diff
```

d displays differences. A patch file records a controlled textual change; patch applies it. Always inspect the patch and make a backup before applying it to operational configuration.

CALCULATION DRILL

Complete the sequence exactly as written. Read each result before continuing.

68. Use `expr` to calculate $29 + 13$.
69. Use `bc` to calculate $355/113$ with `scale=8`.
70. Create a two-line text file and inspect its bytes with `od -c`.
71. Copy the file, change one line with `v`, and compare the versions with `d`.

CHAPTER 14

Networks

Identify interfaces, trace paths, and test services

YANS links terminals, station networks, remote vaults, and surviving long-range relays. Network diagnosis should proceed from local configuration outward: interface, route, reachability, name resolution, service connection, path quality, and packet evidence.

Local identity and routes

COMMAND	PURPOSE	PATTERN	EXAMPLE
i	brief interface addresses	i	i
ifconfig	detailed legacy interface view	ifconfig [interface]	ifconfig eth0
r	routing table	r	r
arp	neighbor cache on local link	arp -n	arp -n
n	active TCP connections and listeners	n	sd n
ss	socket statistics	ss -lntp	sd ss -lntp

Reachability with png

SHORT REACHABILITY TEST

```
$ png archive-gw
PING archive-gw (10.42.0.1) 16(44) bytes of data.
24 bytes from 10.42.0.1: icmp_seq=1 ttl=64 time=0.71 ms
2 packets transmitted, 2 received, 0% packet loss
```

png sends two small probes and summarizes packet loss. A failed probe does not always mean the host is down; policy may block probes while allowing application traffic.

Name resolution

NAMES AND REGISTRATION

```
$ host archive-gw
archive-gw has address 10.42.0.1
$ dig archive-gw
$ whois 203.0.113.40
```

host gives a concise lookup. dig exposes full resolver details. whois queries registration databases when the external network and policy permit.

Test web and certificate paths

HTTP CHECKS

```
$ curl -I https://archive-gw/status
HTTP/2 200
content-type: text/plain
$ cu archive-gw
< HTTP/2 200
* SSL certificate verify ok.
```

curl can retrieve data, send requests, and show headers. cu is a short HTTPS status and certificate summary for one host name.

Remote sessions and file transfer

REMOTE WORK

```
$ s apprentice@relay-4
$ scp report.txt apprentice@relay-4:/home/apprentice/
$ rsync -av reports/ apprentice@relay-4:reports/
```

s opens a secure shell session. scp copies files. rsync synchronizes trees efficiently and can resume partial work. A trailing slash on reports/ means “copy the contents of this directory.”

Path diagnosis

ROUTE QUALITY

```
$ mtr archive-gw
$ mtr -t archive-gw
```

mtr reports the path and loss statistics. Its normal dsh form runs a short report; -t enters a live terminal view. ftp and telnet may exist for legacy equipment but transmit information without modern protection and should only be used on isolated, authorized networks.

Ports and packets

COMMAND	PURPOSE	PATTERN	EXAMPLE
nc	test or open a raw TCP/UDP connection	nc -vz <host> <port>	nc -vz archive-gw 443
nmap	enumerate authorized hosts and ports	nmap <target>	nmap 10.42.0.1
sc	relay sockets and byte streams	sc <addresses>	sc - TCP:host:port
tcpdump	capture packet headers	sd tcpdump -ni <if>	sd tcpdump -ni eth0 port 53

AUTHORIZATION BOUNDARY

Port scanning and packet capture reveal infrastructure and may affect fragile equipment. Use nmap and tcpdump only on systems and networks assigned to your maintenance scope.

NETWORK LADDER

Complete the sequence exactly as written. Read each result before continuing.

72. Run i and identify one interface and address.
73. Run r and identify the default route.
74. Run png against an authorized gateway.
75. Resolve the gateway with host and dig.
76. Use nc -vz to test one documented service port.
77. Use cu on an authorized HTTPS host.

CHAPTER 15

Encoding and encryption

Represent bytes safely and protect secrets deliberately

Encode with bs and decode with bsd

BASE64 REPRESENTATION

```
$ p "Y-2990" | bs
WS0yOTkwCg==
$ p "WS0yOTkwCg==" | bsd
Y-2990
```

bs converts bytes to a text representation that travels safely through text-only channels. bsd reverses the representation. This is encoding, not secrecy: anyone can decode it.

Encrypt with enc

SYMMETRIC ENCRYPTION

```
$ enc access-plan.txt
Enter passphrase:
Repeat passphrase:
$ l access-plan.txt.gpg
```

enc creates an armored encrypted file with the .gpg suffix. The original remains. Transfer or archive the encrypted copy only after confirming that dec can restore a test copy.

Decrypt with dec

DECRYPTION TEST

```
$ mcd decrypt-test
$ cp ../access-plan.txt.gpg .
$ dec access-plan.txt.gpg
Enter passphrase:
$ c access-plan.txt
```

PASSPHRASE RULE

A forgotten passphrase means lost data. A written passphrase can mean exposed data. Follow City key-custody procedure; never place a passphrase in a command line, script, history file, or unencrypted note.

Low-level copying with dd

dd copies raw blocks and is used for devices, images, and controlled recovery. Its source is if= and destination is of=. Reversing them can destroy a storage device.

IMAGE COPY

```
$ dd if=sector.img of=sector-copy.img bs=4M status=progress
```

EXTREME HAZARD

Never use dd against /dev devices until a senior operator has independently verified both paths. Ordinary file-to-file examples are safe training; device imaging is not.

PROTECTION DRILL

Complete the sequence exactly as written. Read each result before continuing.

78. Create a short file named token.txt.
79. Encode it with bs and redirect the result to token.b64.
80. Decode token.b64 with bsd and compare the result.
81. Encrypt token.txt with enc.
82. Copy the encrypted file to a new directory and decrypt it there with dec.
83. Compare the restored text with the original using d.

CHAPTER 16

Field exercise: restore the hydroponics report

A complete workflow using directories, v, filters, archives, processes, and networks

SCENARIO

Hydroponics Station H-12 reports intermittent nutrient alarms. You receive three text logs and a crew table. Your task is to create an evidence directory, identify repeated alarms, write a concise report, archive it, encrypt the archive, and transfer it to the authorized relay.

1. Establish a safe workspace

WORKSPACE

```
$ cd
$ mcd incidents/H12-2990-207
$ o
/home/apprentice/incidents/H12-2990-207
$ md raw report archive
```

Copy the supplied files into raw without changing the originals.

INTAKE

```
$ cp -i /srv/intake/H12/*.log raw/
$ cp -i /srv/intake/H12/crew.csv raw/
$ tree .
```

2. Inspect before processing

INSPECTION

```
$ file raw/*
$ wc -l raw/*.log
$ h -5 raw/nutrients.log
$ t -20 raw/events.log
```

Record the machine time and your identity for traceability.

PROVENANCE

```
$ date > report/provenance.txt
$ id >> report/provenance.txt
$ o >> report/provenance.txt
```

3. Extract alarms

SELECT ALARMS

```
$ g -h ALERT raw/*.log > report/alerts.txt
$ wc -l report/alerts.txt
$ h report/alerts.txt
```

Assume each alarm line contains time, severity, device, and message. Count alarms by device:

COUNT DEVICES

```
$ awk '{print $3}' report/alerts.txt | sort | uniq -c | sort -nr | tee report/alarm-counts.txt
14 mixer-2
5 probe-7
1 pump-1
```

4. Correlate crew assignment

CREW TABLE

```
$ awk -F, 'NR>1 {print $1, $2, $3}' raw/crew.csv | sort > report/crew.txt
$ c report/crew.txt
```

5. Write the human report

OPEN REPORT

```
$ v report/summary.txt
```

Write a short report with these headings: INCIDENT, EVIDENCE, FINDINGS, ACTION, OPERATOR. Refer to exact files and counts. Do not claim a cause that the evidence does not establish.

EVIDENCE LANGUAGE

Distinguish observation from inference. “Fourteen mixer-2 alarms occurred” is an observation. “Mixer-2 has failed” is an inference requiring confirmation.

6. Review and package

REVIEW AND ARCHIVE

```
$ c report/summary.txt
$ d /srv/templates/H12-required-headings.txt <(g "[A-Z][A-Z]*$" report/summary.txt)
$ tar -czvf archive/H12-2990-207.tgz report raw
$ tar -tzf archive/H12-2990-207.tgz | h -30
```

The process-substitution form shown above is available on standard dsh terminals. When a restricted shell lacks it, store the selected headings in a temporary file and compare that file instead.

7. Encrypt and transfer

PROTECT AND TRANSFER

```
$ enc archive/H12-2990-207.tgz
$ l -lh archive/
$ png relay-4
$ nc -vz relay-4 22
$ scp archive/H12-2990-207.tgz.gpg apprentice@relay-4:/srv/intake/hydroponics/
```

8. Verify

REMOTE VERIFICATION

```
$ s apprentice@relay-4
relay-4 $ l -lh /srv/intake/hydroponics/H12-2990-207.tgz.gpg
relay-4 $ x
```

After-action questions

- 84. Which commands changed data, and which only inspected it?
- 85. At which points did you preserve an original copy?
- 86. Which pipeline stage converted raw lines into a device count?
- 87. What evidence would be required before declaring mixer-2 defective?
- 88. Why was the archive inspected before encryption and transfer?

CHAPTER 17

Quick command reference

Core Unix and dsh commands used in this primer

COMMAND	PURPOSE	PATTERN	EXAMPLE
o	show current directory	<code>o</code>	<code>o</code>
l	list directory	<code>l [options] [path]</code>	<code>l -la</code>
cd	change directory	<code>cd [path]</code>	<code>cd logs</code>
md	make directory	<code>md [-p] path</code>	<code>md reports</code>
mcd	make directory and enter it	<code>mcd path</code>	<code>mcd work/new</code>
rd	remove empty directory	<code>rd path</code>	<code>rd old</code>
touch	create/update file timestamp	<code>touch file</code>	<code>touch log</code>
c	display file contents	<code>c file...</code>	<code>c report.txt</code>
v	edit text	<code>v file</code>	<code>v report.txt</code>
cp	copy	<code>cp [options] source dest</code>	<code>cp -i a b</code>
mv	move or rename	<code>mv [options] source dest</code>	<code>mv old new</code>
rm	remove file	<code>rm [options] file</code>	<code>rm -i old</code>
ln	create link	<code>ln [-s] target name</code>	<code>ln -s rel/12 current</code>
file	identify file type	<code>file name</code>	<code>file relay</code>
stat	show metadata	<code>stat name</code>	<code>stat report</code>
h	first lines	<code>h [-n] file</code>	<code>h -20 log</code>
t	last lines/follow	<code>t [-n -f] file</code>	<code>t -f log</code>
le	paged text viewer	<code>le file</code>	<code>le log</code>
b	formatted viewer	<code>b file</code>	<code>b code.c</code>
g	select matching lines	<code>g [options] pattern file</code>	<code>g -n ERROR log</code>
f	find paths	<code>f start tests</code>	<code>f . -name "*.log"</code>
sort	sort lines	<code>sort [options] file</code>	<code>sort -n data</code>
uniq	collapse adjacent duplicates	<code>uniq [options]</code>	<code>sort x uniq -c</code>
cut	select columns/fields	<code>cut options file</code>	<code>cut -d, -f2 crew</code>
paste	combine files by line	<code>paste a b</code>	<code>paste ids names</code>
tr	translate/delete characters	<code>tr set1 set2</code>	<code>tr a-z A-Z</code>
sed	stream editing language	<code>sed expression file</code>	<code>sed 's/a/b/' x</code>
awk	field-processing language	<code>awk program file</code>	<code>awk '{print \$1}' x</code>
wc	count lines/words/bytes	<code>wc options file</code>	<code>wc -l log</code>
tee	display and save input	<code>tee [-a] file</code>	<code>g A log tee a</code>
df	filesystem capacity	<code>df -h</code>	<code>df -h</code>
du	path size	<code>du -sh path</code>	<code>du -sh logs</code>
tar	create/extract archives	<code>tar options archive paths</code>	<code>tar -czvf a.tgz d</code>
gzip	compress one file	<code>gzip [-d] file</code>	<code>gzip data</code>
ps	list processes	<code>ps [options]</code>	<code>ps -ef</code>

COMMAND	PURPOSE	PATTERN	EXAMPLE
top	live process view	top	top
wa	repeat a command	wa command	wa df -h
z	terminate a process	z PID	z 4182
chm	change permissions	chm mode file	chm 640 secret
cho	change ownership	cho owner:group file	sd cho a:ops f
sd	run one command with authority	sd command	sd c secure.log
sb	open administrative shell	sb	sb

Network reference

COMMAND	PURPOSE	PATTERN	EXAMPLE
i	brief interfaces	i	i
r	routes	r	r
n	TCP connections/listeners	n	sd n
png	short reachability test	png host	png gateway
host	concise DNS lookup	host name	host relay-4
dig	detailed DNS query	dig name	dig relay-4
curl	transfer or test URL	curl options URL	curl -I https://host
cu	HTTPS and certificate summary	cu host	cu relay-4
s	secure remote shell	s user@host	s a@relay-4
scp	secure copy	scp source user@host:path	scp r a@h:/tmp/
rsync	synchronize trees	rsync -av source dest	rsync -av r/ a@h:r/
nc	test/connect port	nc -vz host port	nc -vz h 443
mtr	path and loss report	mtr [-t] host	mtr gateway
nmap	authorized port survey	nmap target	nmap 10.0.0.1
tcpdump	authorized packet capture	sd tcpdump filters	sd tcpdump -ni eth0

CHAPTER 18

v editor card

Keep this page beside the terminal during the first week

COMMAND	PURPOSE	PATTERN	EXAMPLE
Esc	return to Normal mode	any uncertainty	Esc Esc
i	insert before cursor	Normal mode	i
a	insert after cursor	Normal mode	a
o	open new line below	Normal mode	o
h j k l	left, down, up, right	Normal mode	j
0	start of line	Normal mode	0
\$	end of line	Normal mode	\$
w	next word	Normal mode	w
b	previous word	Normal mode	b
x	delete character	Normal mode	x
dw	delete word	Normal mode	dw
dd	delete line	Normal mode	dd
yy	copy line	Normal mode	yy
p	paste after cursor/line	Normal mode	p
u	undo	Normal mode	u
Ctrl-R	redo	Normal mode	Ctrl-R
/text	search forward	Normal mode	/ALERT
n	next match	Normal mode	n
N	previous match	Normal mode	N
:w	write	command line	:w
:q	quit	command line	:q
:wq	write and quit	command line	:wq
:q!	discard changes and quit	command line	:q!
:set number	show line numbers	command line	:set number

ONE-WEEK OBJECTIVE

Do not memorize everything. Memorize Esc, i, movement, :w, :q, /, n, dd, and u. The remaining commands will become natural through use.

CHAPTER 19

Operator safety card

Ten rules that prevent most terminal incidents

1. Run `o` before a destructive command in an unfamiliar session.
2. Run `l` on source and destination before `cp`, `mv`, `rm`, tar extraction, or `dd`.
3. Use `-i` with `cp`, `mv`, and `rm` while learning.
4. Make a backup before editing operational configuration.
5. Build pipelines one stage at a time and inspect intermediate output.
6. Quote file names, but prefer operational names without spaces.
7. Prefer one `sd` command over an extended `sb` session.
8. Confirm PID, owner, and command before using `z`.
9. Use network discovery and packet capture only within assigned scope.
10. Test restoration, not merely creation: an archive or encrypted file is useful only when it can be read back.

Emergency stop keys

COMMAND	PURPOSE	PATTERN	EXAMPLE
Ctrl-C	request that foreground command stop	running command	<code>t -f log -> Ctrl-C</code>
q	leave most paged or interactive viewers	<code>le</code> , <code>b</code> , <code>m</code> , <code>top</code>	<code>q</code>
Esc Esc	return <code>v</code> to Normal mode	<code>inside v</code>	<code>Esc Esc</code>
:q!	leave <code>v</code> and discard unsaved changes	<code>inside v</code>	<code>:q!</code>
x	leave the shell	at prompt	<code>x</code>

When you suspect data loss or a security incident, stop issuing commands, preserve the terminal output, record the exact time with date, and contact the responsible City operator.

CHAPTER 20

Issue notes and provenance

Training artifact record

Background: Civilization Y

dsh, or darkshell, is a small Unix-style shell environment imagined as a fragment of a much older future: the command interface of Civilization Y — Yotta.

In the twenty-first century, a nuclear world war destroyed most of human civilization. Nation-states vanished, global networks collapsed, and the accumulated machinery of the old world became either radioactive debris or half-understood myth. The few surviving groups endured in sealed shelters, underground facilities, mountain enclaves, and isolated technical colonies. Over centuries, one of these settlements grew into the City — known simply by that name because, for its inhabitants, there was no other city left.

The City became the last organized human colony on Earth.

Outside its walls lay the cold zones, the ruins, the contaminated industrial belts, and the territories of those who lived beyond civic order. Inside, humanity survived through discipline, rationing, technical continuity, and an almost religious respect for working systems. The old graphical interfaces of the pre-war world were gradually abandoned. They required too much power, too much visual complexity, too many opaque layers. The City's engineers returned to the most durable interface humanity had ever built: text.

By the year 2990, the universal operating system of Civilization Y is Dunix — a distant descendant of Unix, Linux, BSD, and countless forgotten post-war operating systems. It is not a museum copy of the old systems, but their cultural and technical heir. Its philosophy is old and severe: small tools, clear text, pipes, files, permissions, processes, terminals. What survived was what could be repaired, transmitted, typed, audited, and understood.

dsh is the shell of that world.

This manual documents the user-facing command vocabulary of darkshell (dsh) for a fictional Dunix environment in Civilization Y, year 2990. Short commands are intentionally presented as native Dunix commands. Their historical implementation lineage is outside the scope of apprentice training.

Primary technical source

Andy Kosela, darkshell (dsh), project repository and .dsh command environment, master branch inspected 24 July 2026.

Pedagogical inspiration

The progressive, tool-composition approach is inspired by the teaching tradition of The Unix Programming Environment by Brian W. Kernighan and Rob Pike. All prose, examples, exercises, fictional details, and organization in this manual are original.

Edition record

COMMAND	PURPOSE	PATTERN	EXAMPLE
Issue	1.0	Date	2990 City calendar / prepared 2026-07-24
Audience	complete beginners	Scope	core terminal, text, process, network, protection
Environment	Dunix with dsh	Authority	City Technical Continuity Directorate
Status	training publication	Distribution	authorized citizens and operators

CIVILIZATION Y

The City preserved text because text can be typed, audited, transmitted over degraded links, copied by hand, and understood when richer interfaces fail. Dsh is the shell of that survival culture.